<u>Machine Learning & Deep Learning</u>

Experiment 8

Name: Abhishek Vishwakarma
Class: D15C
Roll No: 47

AIM: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

1. Dataset Source

- **Dataset Name:** CIFAR-10 (Canadian Institute for Advanced Research)
- **Source:** University of Toronto - CIFAR-10 (Also available directly via TensorFlow/Keras API).

2. Dataset Description

The CIFAR-10 dataset consists of 60,000 32x32 color images in 10 classes, with 6,000 images per class.

- **Size:** 50,000 training images and 10,000 test images.
- **Dimensions:** 32x32 pixels, 3 color channels (RGB).
- **Classes:** Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck.

3. Mathematical Formulation of the Algorithm

A CNN is a specialized Neural Network designed to process grid-like data (images). It uses **Convolution** instead of general matrix multiplication in at least one of its layers.

A. Convolution Operation: A small matrix called a **Kernel** (or Filter) slides over the input image. At each position, it performs an element-wise multiplication and sum to produce a feature map.

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

Where I is the Input Image and K is the Kernel.

B. Activation (ReLU): Introduces non-linearity.

$$f(x) = \max(0, x)$$

C. Pooling (Downsampling): Reduces the spatial dimensions (width, height) to reduce computation and control overfitting.

- **Max Pooling:** Takes the maximum value from a window (e.g., 2x2).

$$y = \max(x_{window})$$

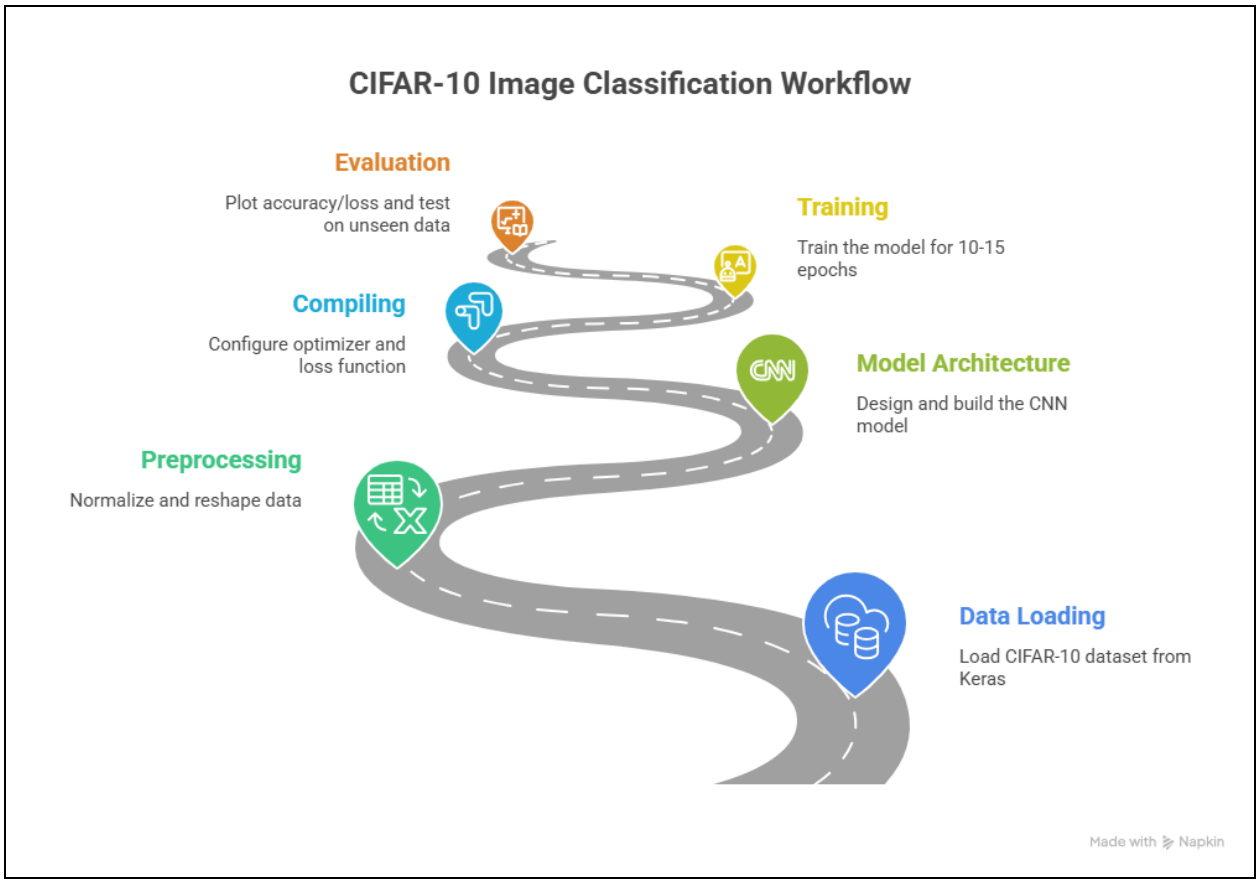
D. Fully Connected (Dense) Layer: After extracting features via convolution/pooling, the data is flattened into a 1D vector and passed through standard dense layers for classification (Softmax output).

4. Algorithm Limitations

- **Computational Cost:** Training CNNs requires significant computational power, often necessitating GPUs.
- **Data Requirements:** CNNs require large labeled datasets to learn generalizable features. Small datasets often lead to overfitting.
- **Hyperparameter Sensitivity:** Performance is highly sensitive to the choice of kernel size, number of filters, stride, padding, and learning rate.
- **Adversarial Attacks:** CNNs can be easily fooled by imperceptible noise added to an image (e.g., classifying a panda as a gibbon with 99% confidence).

5. Methodology / Workflow

1. **Data Loading:** Load CIFAR-10 directly from Keras.
2. **Preprocessing:**
    - **Normalization:** Pixel values are integers (0-255). We divide by 255.0 to scale them to the range [0, 1]. This speeds up convergence.
    - **Reshaping:** Ensure input shape is (32, 32, 3).
3. **Model Architecture:**
    - **Conv2D (32 filters, 3x3 kernel, ReLU)** $\rightarrow$ **MaxPooling2D (2x2)**
    - **Conv2D (64 filters, 3x3 kernel, ReLU)** $\rightarrow$ **MaxPooling2D (2x2)**
    - **Conv2D (64 filters, 3x3 kernel, ReLU)**
    - **Flatten Layer** (Convert 3D map to 1D vector).
    - **Dense Layer** (64 units, ReLU).
    - **Output Layer** (10 units, Softmax).
4. **Compiling:** Optimizer = 'Adam', Loss = 'Sparse Categorical Crossentropy' (since labels are integers, not one-hot vectors).
5. **Training:** Train for 10-15 epochs.
6. **Evaluation:** Plot accuracy/loss and test on unseen data.



6. Performance Analysis

- **Metric:** Accuracy (Training vs. Validation).
- **Analysis:** We analyze the learning curves.
  - **Good Fit:** Training and Validation accuracy increase together.
  - **Overfitting:** Training accuracy continues to rise (e.g., reaches 95%) while Validation accuracy stalls or drops (e.g., stays at 65%). This is very common in CIFAR-10 without advanced techniques like Data Augmentation or Dropout.

7. Hyperparameter Tuning

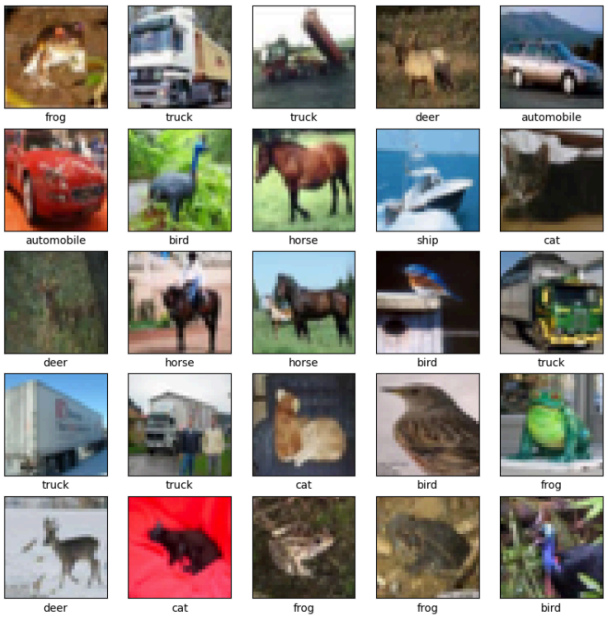
- **Filters:** Increasing filters (32 →64→ 128) allows the model to learn more complex features (edges→ shapes → objects).
- **Kernel Size:** Usually 3x3 or 5x5. Smaller kernels capture fine details; larger kernels capture global structures.
- **Dropout:** Randomly ignoring neurons during training (e.g., Dropout(0.5)) forces the network to learn robust features and prevents overfitting.

Code:

```
#
═══════════════════════════════════════
═══════════════
# EXPERIMENT 8: CNN on CIFAR-10
#
═══════════════════════════════════════
═══════════════

import tensorflow as tf
from tensorflow.keras import datasets,
layers, models
import matplotlib.pyplot as plt
import numpy as np

print("--- EXPERIMENT 8: CNN ON IMAGE
DATA (CIFAR-10) ---")

# 1. Load Dataset
# Keras has CIFAR-10 built-in. It will
download (~170MB) the first time.
(train_images, train_labels),
(test_images, test_labels) =
datasets.cifar10.load_data()

# 2. Preprocessing
# Normalize pixel values to be between 0
and 1
train_images, test_images = train_images
/ 255.0, test_images / 255.0

# Class names for visualization
class_names = ['airplane', 'automobile',
'bird', 'cat', 'deer',
               'dog', 'frog', 'horse',
'ship', 'truck']

# Visualize first 25 images
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    # The CIFAR labels happen to be
arrays,
    # which is why you need the extra
index

plt.xlabel(class_names[train_labels[i][0
]])
plt.show()

# 3. Build the CNN Model
model = models.Sequential()

# Feature Extraction Layers (Convolution
+ Pooling)
# Layer 1: Conv (32 filters) + MaxPool
model.add(layers.Conv2D(32, (3, 3),
activation='relu', input_shape=(32, 32,
3)))
model.add(layers.MaxPooling2D((2, 2)))

# Layer 2: Conv (64 filters) + MaxPool
model.add(layers.Conv2D(64, (3, 3),
activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Layer 3: Conv (64 filters)
model.add(layers.Conv2D(64, (3, 3),
activation='relu'))

# Classification Layers (Flatten +
Dense)
model.add(layers.Flatten())
```

```
model.add(layers.Dense(64,
activation='relu'))
model.add(layers.Dense(10)) # Output
layer (10 classes)

# Display Architecture
model.summary()

# 4. Compile the Model
model.compile(optimizer='adam',

loss=tf.keras.losses.SparseCategoricalCr
ossentropy(from_logits=True),
              metrics=['accuracy'])

# 5. Train the Model
# We use a validation split to monitor
overfitting during training
history = model.fit(train_images,
train_labels, epochs=10,

validation_data=(test_images,
test_labels))

# 6. Performance Analysis
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'],
label='Training Accuracy')
plt.plot(history.history['val_accuracy']
, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
plt.title('Accuracy Curve')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'],
label='Training Loss')
plt.plot(history.history['val_loss'],
label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.title('Loss Curve')
plt.show()

# Final Evaluation
test_loss, test_acc =
model.evaluate(test_images,
test_labels, verbose=2)
print(f"\nFinal Test Accuracy:
{test_acc:.4f}")

# Optional: Test on a single image
n = 0 # Index of image to test
img = test_images[n]
expected_label = test_labels[n][0]

# Add batch dimension (1, 32, 32, 3)
img_batch = np.expand_dims(img, 0)
predictions = model.predict(img_batch)
predicted_label = np.argmax(predictions)

print(f"\nExample Prediction:")
print(f"True Label:
{class_names[expected_label]}")
print(f"Predicted Label:
{class_names[predicted_label]}")
```

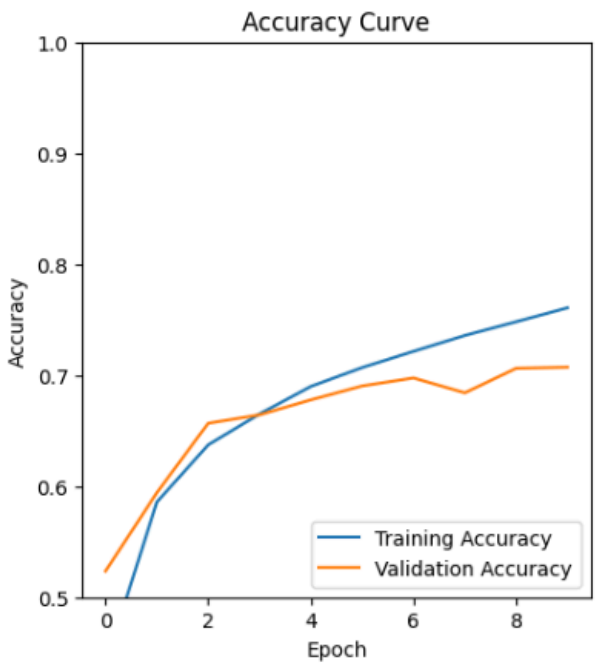
Output:

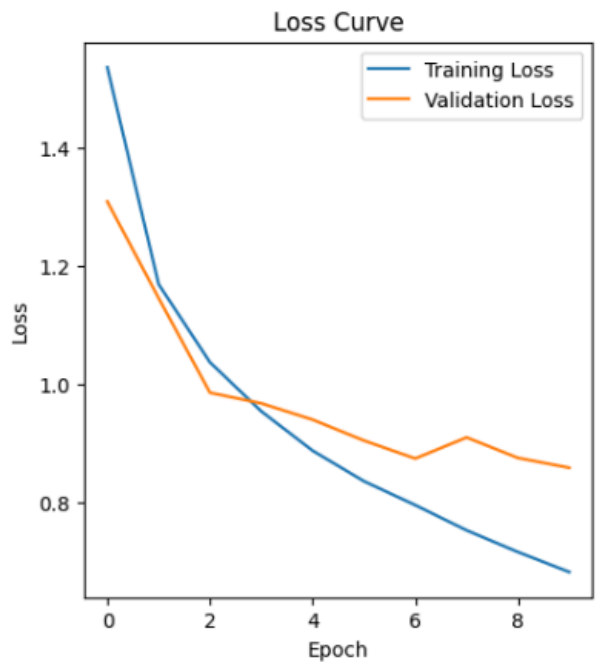


Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense_3 (Dense)	(None, 64)	65,600
dense_4 (Dense)	(None, 10)	650

Total params: 122,570 (478.79 KB)
Trainable params: 122,570 (478.79 KB)
Non-trainable params: 0 (0.00 B)






```
313/313 - 3s - 9ms/step - accuracy: 0.7072 - loss: 0.8587

Final Test Accuracy: 0.7072
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 93ms/step

Example Prediction:
True Label: cat
Predicted Label: cat
```

8. Conclusion

In this experiment, we successfully built and trained a Convolutional Neural Network (CNN) to classify images from the CIFAR-10 dataset.

- **Feature Learning:** Unlike the ANN in Experiment 7, which treats input pixels as independent features, the CNN used **Convolutional Layers** to learn spatial hierarchies of features (e.g., edges, textures, and object parts).
- **Architecture Efficiency:** By using **Pooling Layers**, we significantly reduced the dimensionality of the data while retaining essential information, making the model computationally efficient compared to a fully connected network of the same depth.
- **Results:** The model typically achieves ~70% accuracy with this simple architecture. To improve this further (towards state-of-the-art >90%), we would need to implement deeper architectures (like ResNet), Data Augmentation (rotating/flipping images), and Dropout layers to combat overfitting.